

Comunicación Serie: Node-RED con NUCLEO-F446RE

1- Dr. Lic. Roberto Federico FARFÁN

farfan.roberto.f@gmail.com

2- Esp. Ing. Luis Mariano CAMPOS

1,2-Profesor de Introducción a los Sistemas Embebidos - **Facultad de Ingeniería – UBA**

1-Profesor de Electrónica, Electrónica Analógica y Digital - **Facultad de Ingeniería - U.N.Sa**

2- **CONICET**

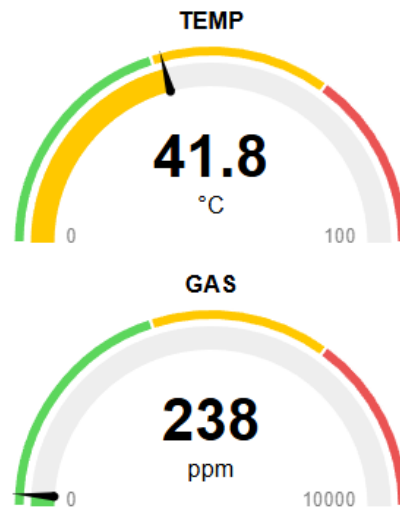
Resumen de la clase:

- 2.1 Temas para la semana 2.
- 2.2 Repaso: USART.
- 2.3 Configuración UART por medio de CUBE MX.
- 2.4 Uso de las herramientas HAL (Hardware Abstraction Layer).
- 2.5 Node-RED.
- 2.6 Problemas: Comunicación Node-RED con NUCLEO-F446RE.

Temas para la semana 2

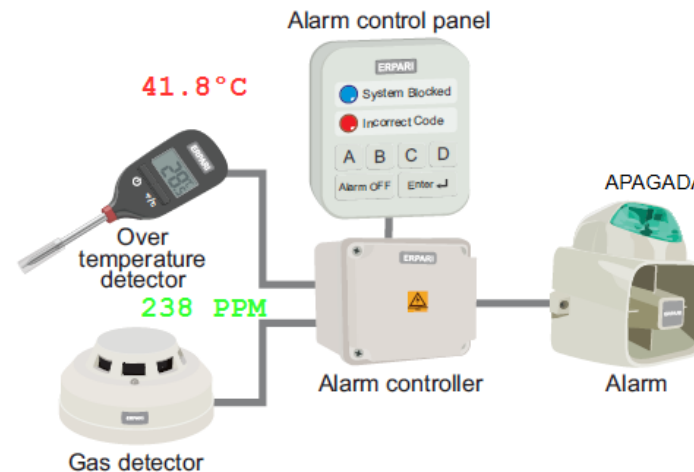
≡ Node-RED y NUCLEO-F446

PruebaADC



Ref: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers

Ejemplo1-1



2.1 Temas para la semana 2.

Semana	Temas de teoría/práctica	Fecha entrega informe TP	Bibliografía básica
1	Contenido de las clases de la semana 1. Teoría: Definición de sistemas embebidos. Procesadores ARM Cortex-M: arquitectura y bloques internos. Concepto de álgebra booleana y entradas/salidas digitales. Entorno STM32CubeIDE. Práctica: Programación básica de GPIO (Entradas/Salidas). Estructura del proyecto de la materia.	Será informada por el docente a cargo.	1, 2, 3 y 4.
2	Contenido de las clases de la semana 2. Teoría: Protocolos de comunicación entre circuitos integrados. Comunicación Serie. <u>Node-RED</u>. Instalación y características de la programación de flujos. Práctica: Configuración del puerto serie. Envío y recepción de datos. Configuración del nodo de comunicación serie en <u>Nodo-RED</u>. Conexión <u>Node-RED</u> y placa NUCLEO-F446RE.	Será informada por el docente a cargo.	1, 2, 3 y 4.

Repaso: USART

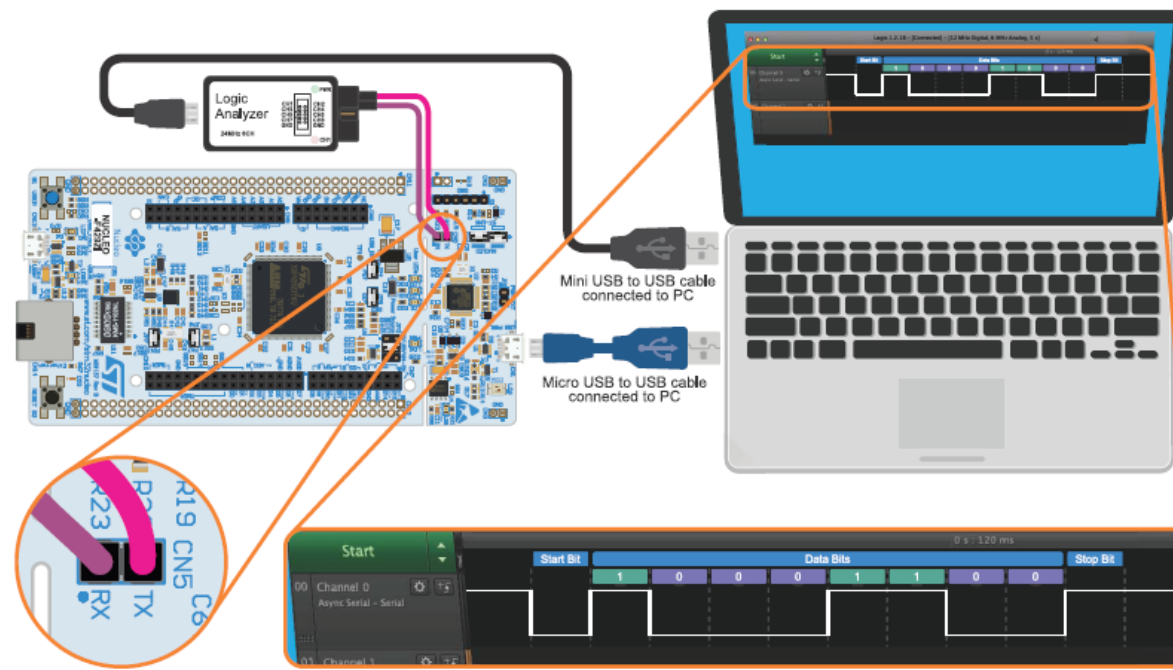


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 82.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

2.2 Repaso: USART.

- Acrónimo de Universal Synchronous and Asynchronous Receiver-Transmitter.
- Es un protocolo de comunicación serie que permite el intercambio de datos de forma síncrona o asíncrona.
- El μ C STM32F446RE dispone de 4 USARTs y 2 UARTs.
- Función Principal: Comunicación con ordenadores u otros dispositivos externos mediante una interfaz serie.

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>



life.augmented

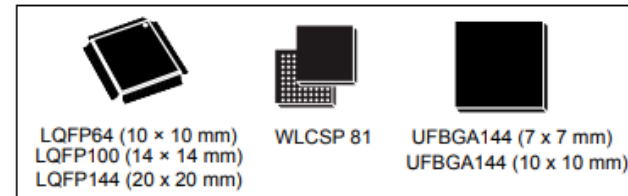
STM32F446xC/E

Arm[®] Cortex[®]-M4 32-bit MCU+FPU, 225 DMIPS, up to 512 KB Flash/128+4 KB RAM, USB OTG HS/FS, seventeen TIMs, three ADCs and twenty communication interfaces

Datasheet - production data

Features

- Core: Arm[®] 32-bit Cortex[®]-M4 CPU with FPU, Adaptive real-time accelerator (ART Accelerator) allowing 0-wait state execution from Flash memory, frequency up to 180 MHz, MPU, 225 DMIPS/1.25 DMIPS/MHz (Dhrystone 2.1), and DSP instructions
- Memories
 - 512 Kbytes of Flash memory
 - 128 Kbytes of SRAM
 - Flexible external memory controller with up to 16-bit data bus: SRAM, PSRAM, SDRAM/LPDDR SDRAM, NOR/NAND Flash memories
 - Dual mode QuadSPI interface
- LCD parallel interface, 8080/6800 modes



- Up to 114 I/O ports with interrupt capability
 - Up to 111 fast I/Os up to 90 MHz
 - Up to 112 5 V-tolerant I/Os
- Up to 20 communication interfaces
 - SPDIF-Rx
 - Up to 4x I²C interfaces (SMBus/PMBus)
 - Up to four USARTs and two UARTs (11.25 Mbit/s, ISO7816 interface, LIN, IrDA, modem control)
 - Up to four SPIs (45 Mbit/s), three with muxed I²S for audio class accuracy via

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>

Table 8. USART feature comparison⁽¹⁾

USART name	Standard features	Modem (RTS/CTS)	LIN	SPI master	IrDA	Smartcard (ISO 7816)	Max. baud rate in Mbit/s		APB mapping
							Oversampling by 16	Oversampling by 8	
USART1	X	X	X	X	X	X	5.62	11.25	APB2 (max. 90 MHz)
USART2	X	X	X	X	X	X	2.81	5.62	APB1 (max. 45 MHz)
USART3	X	X	X	X	X	X	2.81	5.62	
UART4	X	X	X	-	X	-	2.81	5.62	
UART5	X	X	X	-	X	-	2.81	5.62	APB2 (max. 90 MHz)
USART6	X	X	X	X	X	X	5.62	11.25	

- Los puertos 1, 2, 3 y 6 pueden funcionar en modo síncrono o asíncrono.
- Los puertos 4 y 5 son puramente asíncronos.

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>

Table 8. USART feature comparison⁽¹⁾

USART name	Standard features	Modem (RTS/CTS)	LIN	SPI master	IrDA	Smartcard (ISO 7816)	Max. baud rate in Mbit/s		APB mapping
							Oversampling by 16	Oversampling by 8	
USART1	X	X	X	X	X	X	5.62	11.25	APB2 (max. 90 MHz)
USART2	X	X	X	X	X	X	2.81	5.62	APB1 (max. 45 MHz)
USART3	X	X	X	X	X	X	2.81	5.62	
UART4	X	X	X	-	X	-	2.81	5.62	
UART5	X	X	X	-	X	-	2.81	5.62	APB2 (max. 90 MHz)
USART6	X	X	X	X	X	X	5.62	11.25	

- Standard Features: Se refiere a las funciones básicas de envío y recepción de datos, generación de bits de parada y detección de errores de paridad.

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>

Table 8. USART feature comparison⁽¹⁾

USART name	Standard features	Modem (RTS/CTS)	LIN	SPI master	IrDA	Smartcard (ISO 7816)	Max. baud rate in Mbit/s		APB mapping
							Oversampling by 16	Oversampling by 8	
USART1	X	X	X	X	X	X	5.62	11.25	APB2 (max. 90 MHz)
USART2	X	X	X	X	X	X	2.81	5.62	APB1 (max. 45 MHz)
USART3	X	X	X	X	X	X	2.81	5.62	
UART4	X	X	X	-	X	-	2.81	5.62	
UART5	X	X	X	-	X	-	2.81	5.62	APB2 (max. 90 MHz)
USART6	X	X	X	X	X	X	5.62	11.25	

- Modem (RTS/CTS): Es el control de flujo por hardware (Hardware Flow Control), permite que el receptor le indique al emisor si el buffer está lleno, mediante cables adicionales.

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>

Table 8. USART feature comparison⁽¹⁾

USART name	Standard features	Modem (RTS/CTS)	LIN	SPI master	IrDA	Smartcard (ISO 7816)	Max. baud rate in Mbit/s		APB mapping
							Oversampling by 16	Oversampling by 8	
USART1	X	X	X	X	X	X	5.62	11.25	APB2 (max. 90 MHz)
USART2	X	X	X	X	X	X	2.81	5.62	APB1 (max. 45 MHz)
USART3	X	X	X	X	X	X	2.81	5.62	
UART4	X	X	X	-	X	-	2.81	5.62	
UART5	X	X	X	-	X	-	2.81	5.62	APB2 (max. 90 MHz)
USART6	X	X	X	X	X	X	5.62	11.25	

- LIN (Local Interconnect Network): Protocolo común en la industria automotriz, todos lo soportan.
- SPI Master: Permite que la USART actúe como un puerto SPI simple, la UART4 y UART5 NO lo soportan.

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>

Table 8. USART feature comparison⁽¹⁾

USART name	Standard features	Modem (RTS/CTS)	LIN	SPI master	irDA	Smartcard (ISO 7816)	Max. baud rate in Mbit/s		APB mapping
							Oversampling by 16	Oversampling by 8	
USART1	X	X	X	X	X	X	5.62	11.25	APB2 (max. 90 MHz)
USART2	X	X	X	X	X	X	2.81	5.62	APB1 (max. 45 MHz)
USART3	X	X	X	X	X	X	2.81	5.62	
UART4	X	X	X	-	X	-	2.81	5.62	
UART5	X	X	X	-	X	-	2.81	5.62	APB2 (max. 90 MHz)
USART6	X	X	X	X	X	X	5.62	11.25	

- irDA (Infrared Data Association): Protocolo para comunicación por infrarrojos.
- Smartcard (ISO 7816): Interfaz para leer tarjetas con chip (como tarjetas de crédito o SIM), UART4 y UART5 NO soportan esto.

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>

Table 8. USART feature comparison⁽¹⁾

USART name	Standard features	Modem (RTS/CTS)	LIN	SPI master	IrDA	Smartcard (ISO 7816)	Max. baud rate in Mbit/s		APB mapping
							Oversampling by 16	Oversampling by 8	
USART1	X	X	X	X	X	X	5.62	11.25	APB2 (max. 90 MHz)
USART2	X	X	X	X	X	X	2.81	5.62	APB1 (max. 45 MHz)
USART3	X	X	X	X	X	X	2.81	5.62	
UART4	X	X	X	-	X	-	2.81	5.62	
UART5	X	X	X	-	X	-	2.81	5.62	APB2 (max. 90 MHz)
USART6	X	X	X	X	X	X	5.62	11.25	

- La velocidad máxima depende de dos factores:
- Oversampling by 16: El receptor toma 16 muestras por cada bit para asegurar que el valor sea correcto (Robusto pero lento (máximo 5.62 Mbit/s en USART1/6)).

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>

Table 8. USART feature comparison⁽¹⁾

USART name	Standard features	Modem (RTS/CTS)	LIN	SPI master	IrDA	Smartcard (ISO 7816)	Max. baud rate in Mbit/s		APB mapping
							Oversampling by 16	Oversampling by 8	
USART1	X	X	X	X	X	X	5.62	11.25	APB2 (max. 90 MHz)
USART2	X	X	X	X	X	X	2.81	5.62	APB1 (max. 45 MHz)
USART3	X	X	X	X	X	X	2.81	5.62	
UART4	X	X	X	-	X	-	2.81	5.62	
UART5	X	X	X	-	X	-	2.81	5.62	APB2 (max. 90 MHz)
USART6	X	X	X	X	X	X	5.62	11.25	

- La velocidad máxima depende de dos factores:
- Oversampling by 8: Toma solo 8 muestras. Permite duplicar la velocidad (hasta 11.25 Mbit/s) pero es más sensible al ruido.

2.2 Datasheet.

- <https://www.st.com/resource/en/datasheet/stm32f446re.pdf>

Table 8. USART feature comparison⁽¹⁾

USART name	Standard features	Modem (RTS/CTS)	LIN	SPI master	IrDA	Smartcard (ISO 7816)	Max. baud rate in Mbit/s		APB mapping
							Oversampling by 16	Oversampling by 8	
USART1	X	X	X	X	X	X	5.62	11.25	APB2 (max. 90 MHz)
USART2	X	X	X	X	X	X	2.81	5.62	APB1 (max. 45 MHz)
USART3	X	X	X	X	X	X	2.81	5.62	
UART4	X	X	X	-	X	-	2.81	5.62	
UART5	X	X	X	-	X	-	2.81	5.62	APB2 (max. 90 MHz)
USART6	X	X	X	X	X	X	5.62	11.25	

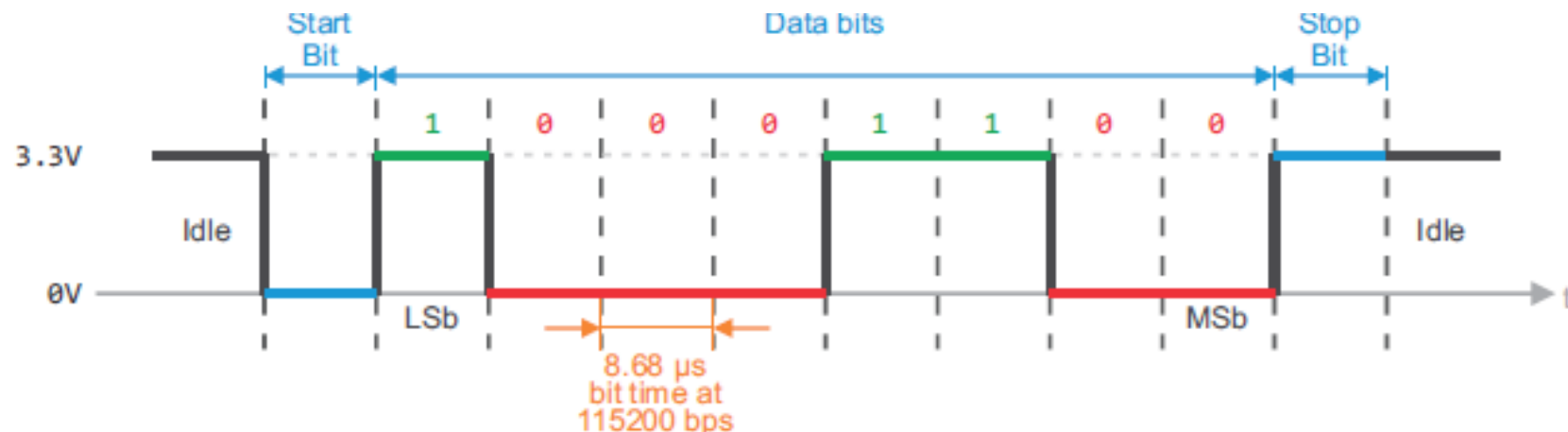
- USART1 y USART6 están conectados al bus APB2, que es el bus de alta velocidad (máximo 90 MHz).
- USART2, USART3, UART4 y UART5 están en el bus APB1, que es más lento (máximo 45 MHz).

2.2 Repaso: Características Principales.

- Comunicación Full-Duplex: Permite enviar y recibir datos simultáneamente a través de líneas separadas (TX y RX).
- Velocidad Programable: Soporta tasas de baudios de hasta **11,25 Mbps en el STM32F446RE**.
- Formato de Trama: Configurable en longitud de datos **(7 u 8 bits)**, bits de parada (1 o 2) y bits de paridad para detección de errores.
- Generación de Interrupciones: El periférico puede avisar a la CPU cuando se recibe un dato o cuando la transmisión ha finalizado.

2.2 Repaso: Conexión Física y Niveles Lógicos.

- Los PCs modernos no tienen **puertos serie nativos**, se utilizan conversores USB a serie (como el CP2102).
- Lógica Inversa: En reposo (estado idle), la línea de comunicación se mantiene en nivel ALTO.



2.2 Repaso: Conexión Física y Niveles Lógicos.

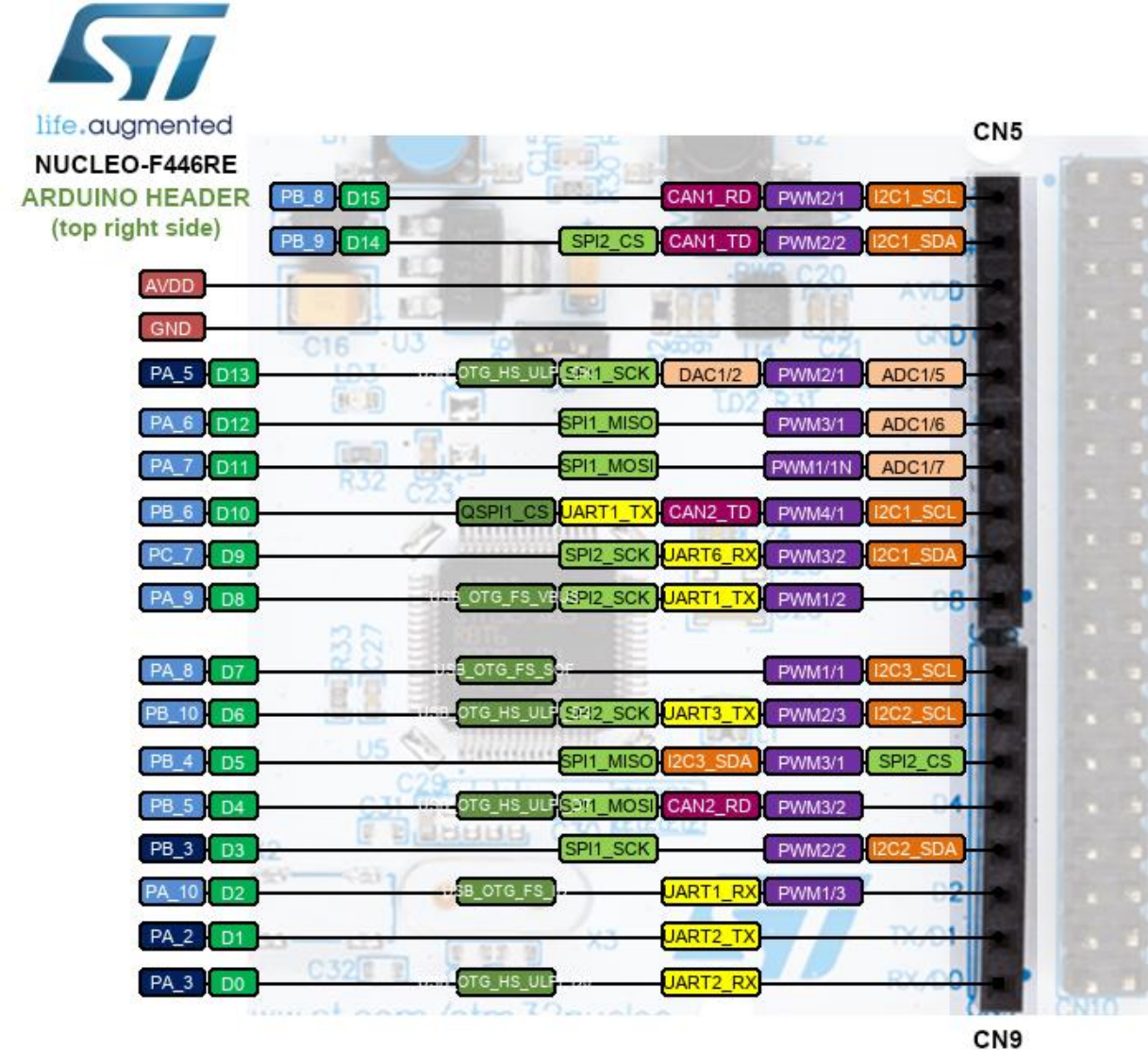


Figure 2.12 Basic setup for a serial communication between two devices.

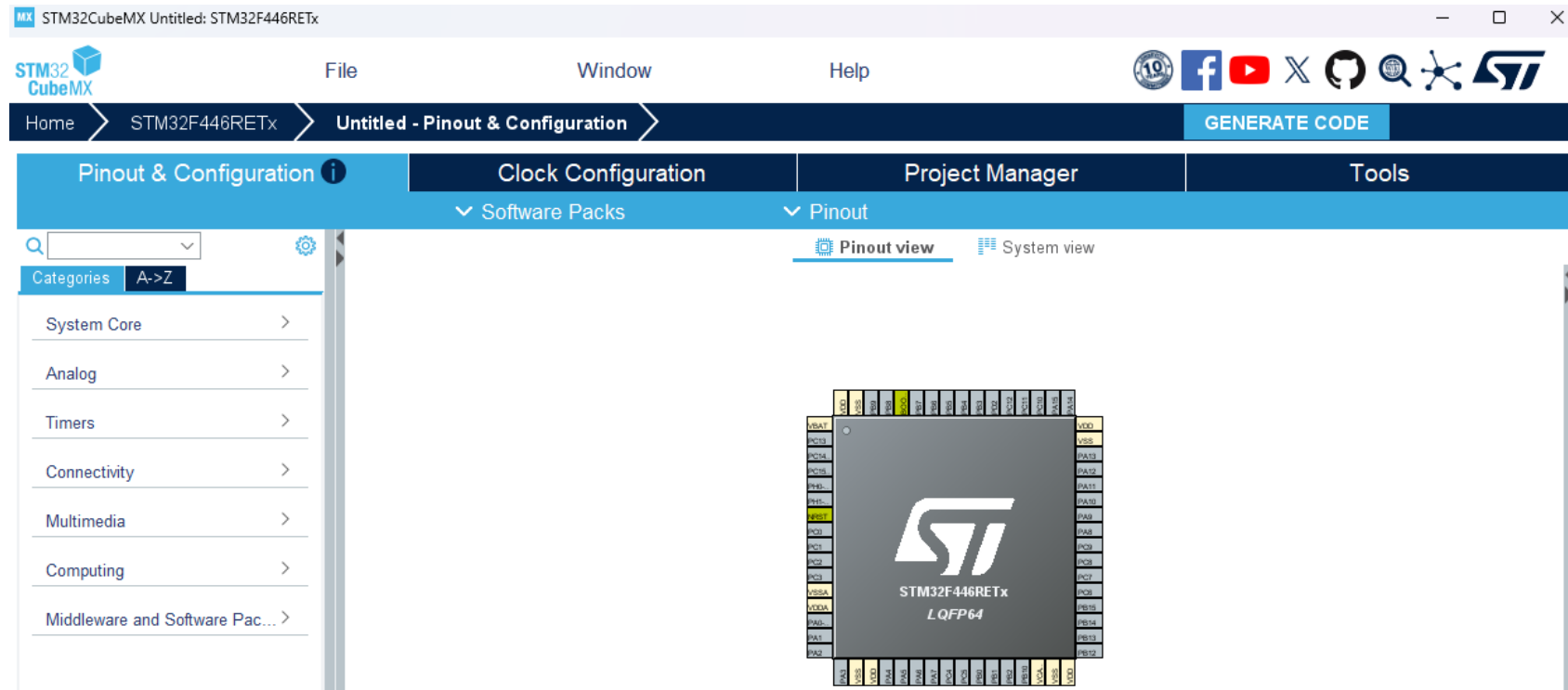
- Bit de Inicio: La transmisión comienza con un bit de inicio (valor '0').
- Conexión Cruzada: El pin TX del microcontrolador debe conectarse al pin RX del receptor, y viceversa.
- ***El emisor y el receptor deben configurarse a la misma tasa de baudios.***

2.2 Repaso: UART2.

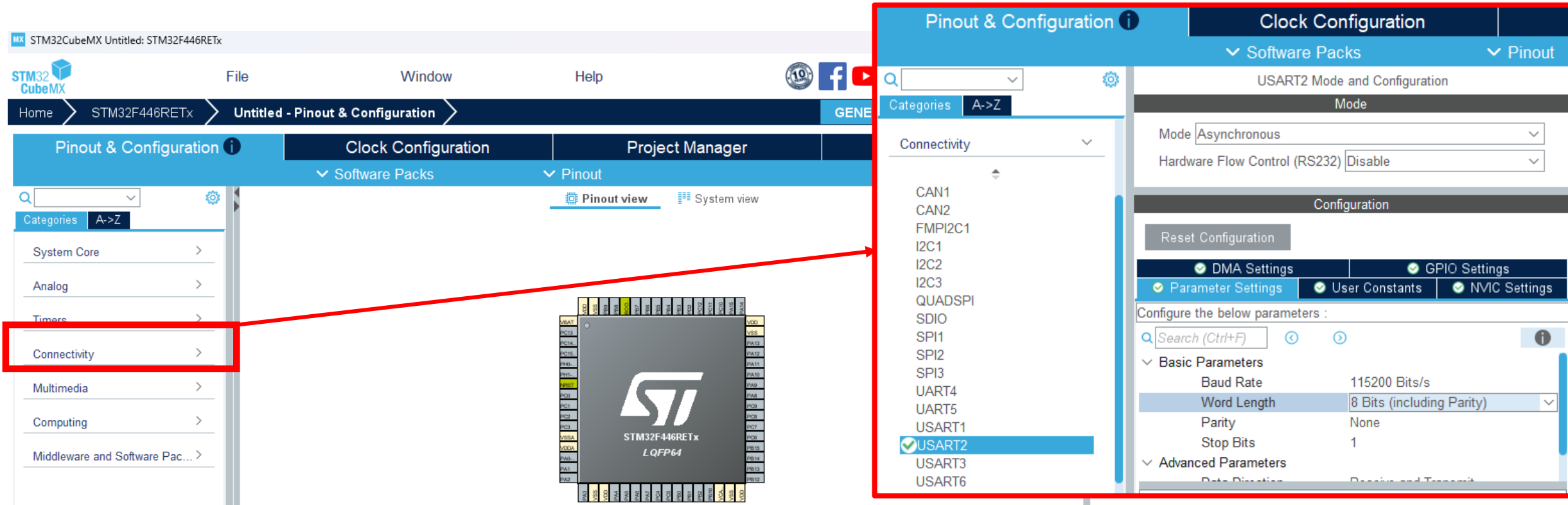
- La USART2 viene conectada por defecto al ST-LINK.
- Permite una comunicación directa con el PC a través del mismo cable USB de programación.
- Se puede acceder desde Tx/D1, Rx/D0 o Tx y Rx.
- <https://os.mbed.com/platforms/>



Configuración UART por medio de CUBE MX

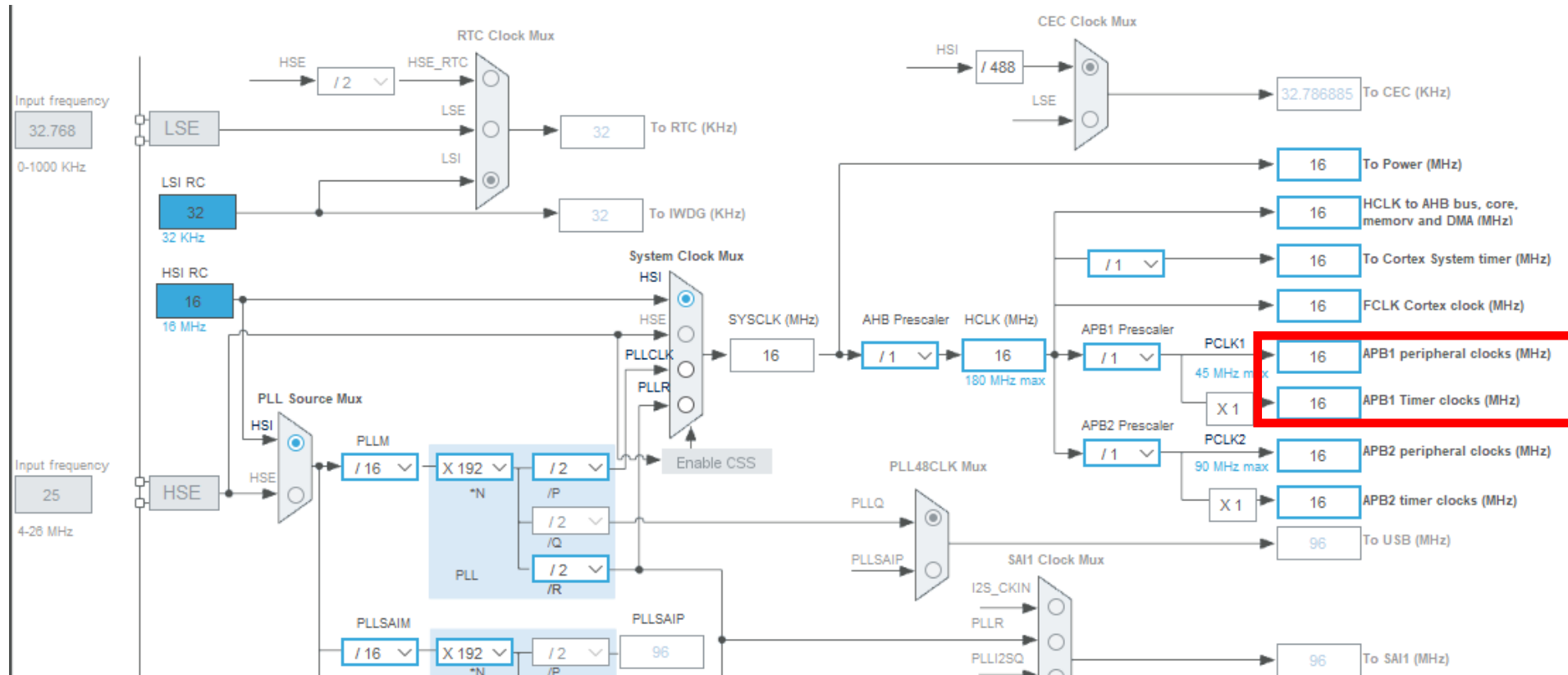


2.3 Configuración UART - CUBE MX.



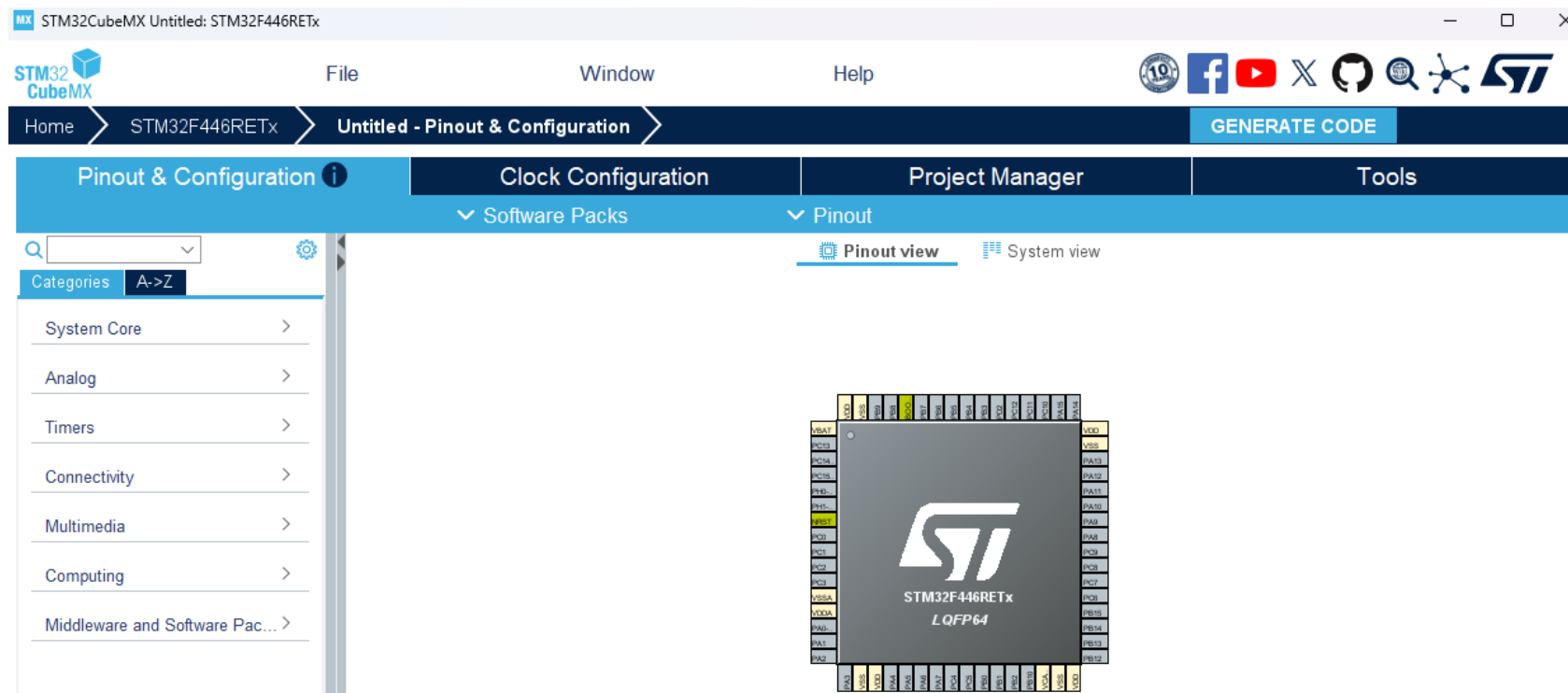
- Buscar Connectivity -> USART2, en Mode se selecciona Asynchronous.
- En Parameter Settings completamos lo siguiente: Baud Rate: 115200, Word Length: 8 bits, Parity: None y Stop Bits: 1.

2.3 Configuración UART - CUBE MX.



- Clock Configuration: Asegúrate de que el reloj del sistema esté configurado, podemos usar 16 MHz.

Uso de las herramientas HAL



2.4 Uso de las herramientas HAL.

```
#include <string.h>
#include "main.h"
#include "usart.h"
#include "gpio.h"

void SystemClock_Config(void);
int main(void) {
    char msg[] = "4\n";
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    while (1) {
        HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
        HAL_Delay(2000);
    }
}
```


2.4 Uso de las herramientas HAL.

- Se declara un `char msg[] = "4\n";`, creando un arreglo de caracteres.
- Función `HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg_uno), 100);`
- `&huart2`: puntero a la estructura que contiene toda la configuración del puerto (Baud rate, pins, registros), indica que utilice USART2.
- `(uint8_t*)msg`, los datos: convierte la cadena de caracteres en una cadena de bytes. La UART envía bytes.

2.4 Uso de las herramientas HAL.

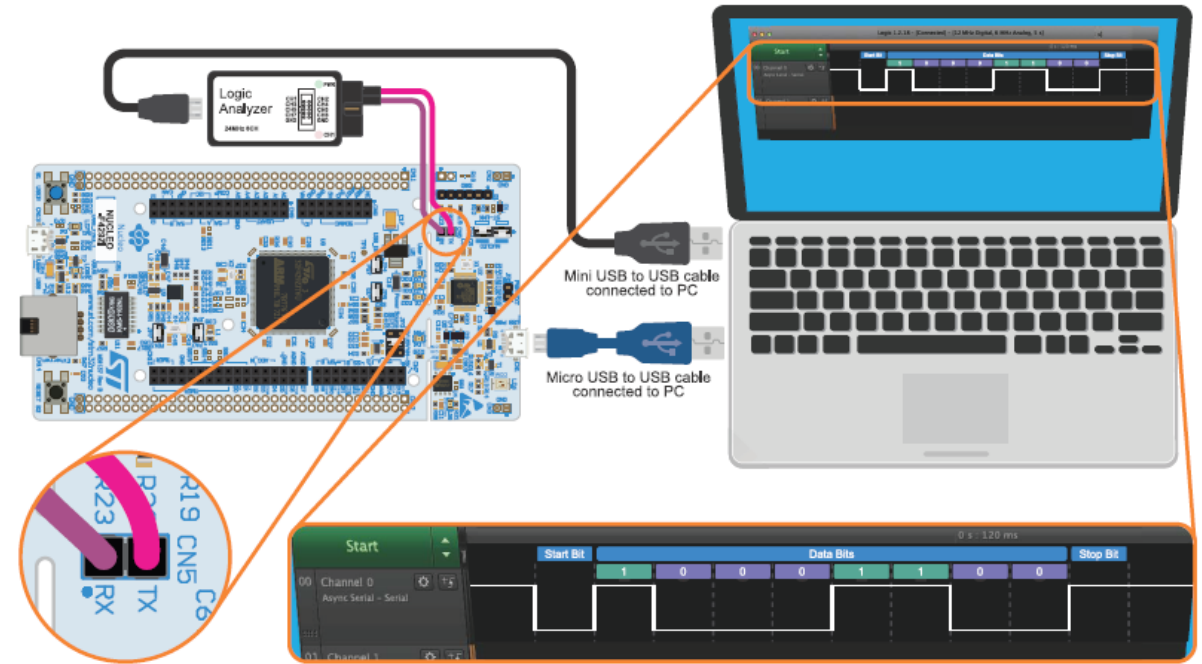
- `strlen(msg)`, tamaño: Indica cuántos bytes debe enviar. `Strlen` va a devolver un 2 (2 bytes).
- **Es necesario definir `#include <string.h>`.**
- 100 (Timeout): Tiempo máximo en milisegundos que la CPU está dispuesta a esperar a que el hardware de la UART termine de enviar los datos (Si no, la función sale para no bloquear el programa).

2.4 Uso de las herramientas HAL.

- ¿Qué información observo?

```
#include <string.h>
#include "main.h"
#include "usart.h"
#include "gpio.h"
```

```
void SystemClock_Config(void);
int main(void){
    char msg[] = "4\n";
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    while (1){
        HAL_UART_Transmit(&huart2,(uint8_t*)msg, strlen(msg), 100);
        HAL_Delay(2000);
    }
}
```

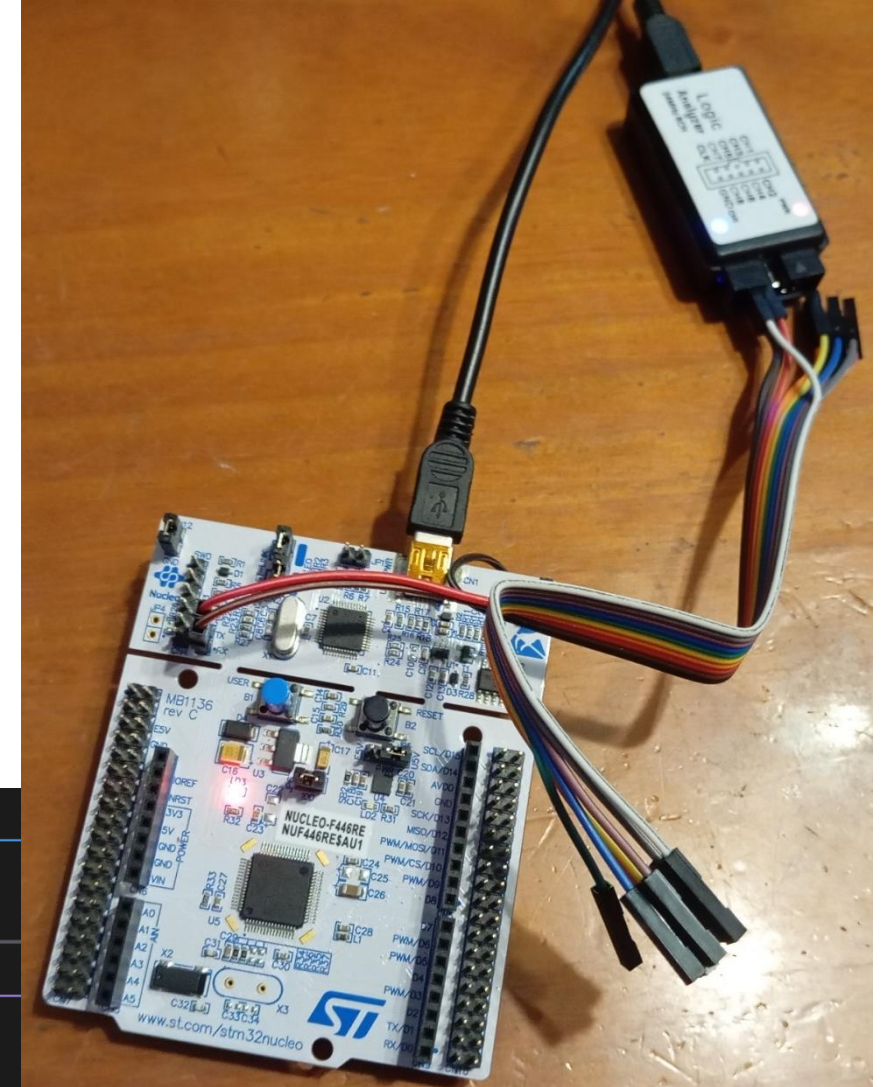
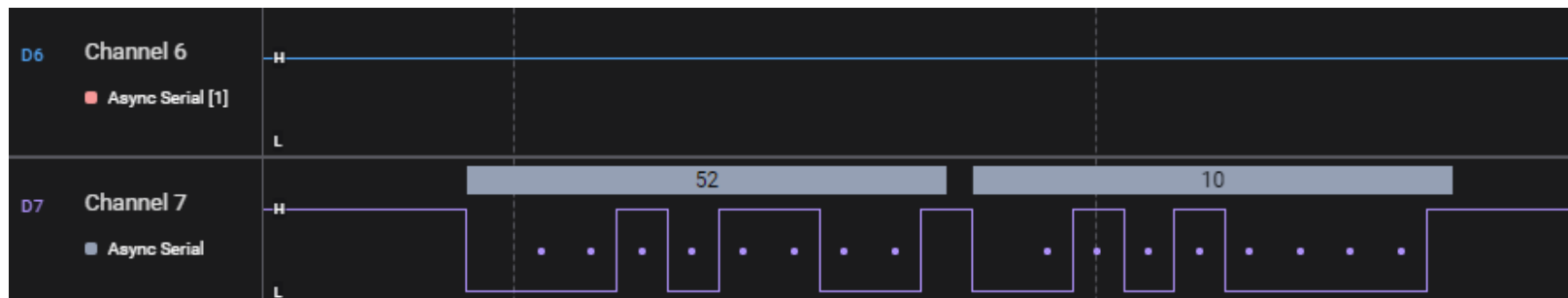


2.4 Uso de las herramientas HAL.

- ¿Qué información observo?

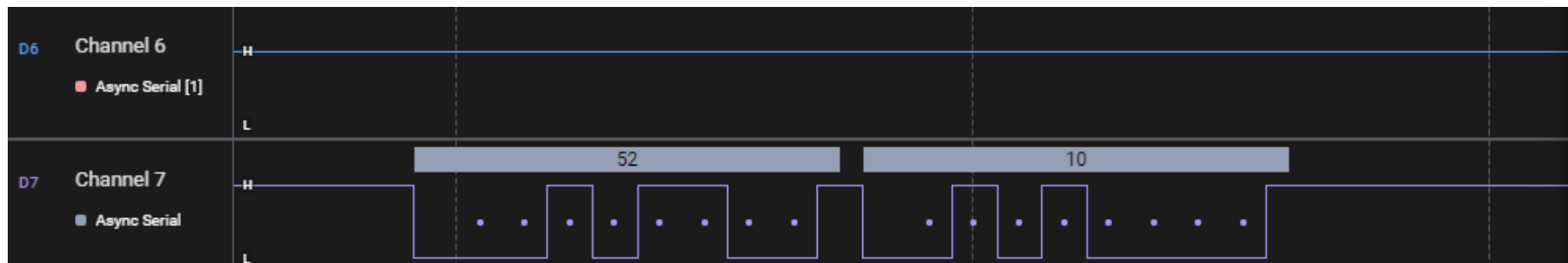
```
#include <string.h>
#include "main.h"
#include "usart.h"
#include "gpio.h"
```

```
void SystemClock_Config(void);
int main(void){
char msg[] = "4\n";
HAL_Init();
SystemClock_Config();
```

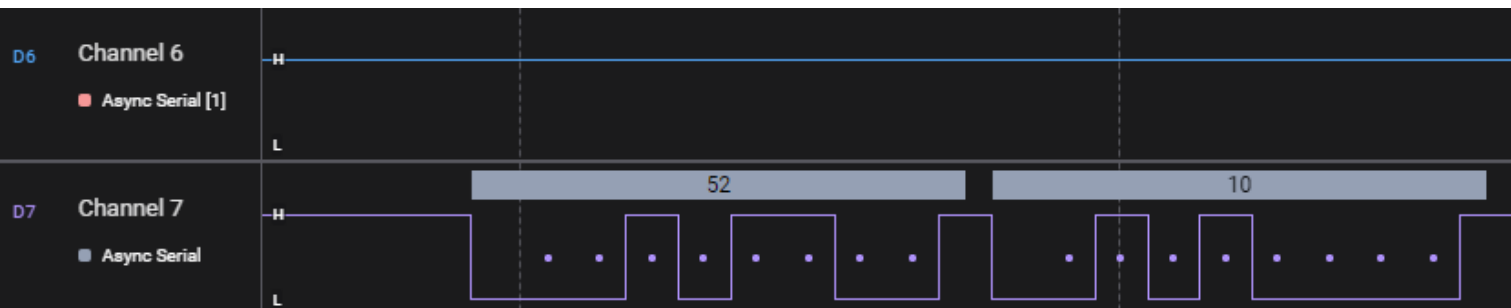


2.4 Uso de las herramientas HAL.

- El número 52: En la tabla ASCII, el carácter '4' corresponde al valor decimal 52.
- El número 10: El carácter `\n` representa un "Line Feed" (Salto de línea). En la tabla ASCII este comando corresponde al 10.
- En modo binario el 52 es una secuencia de bits: 00110100.

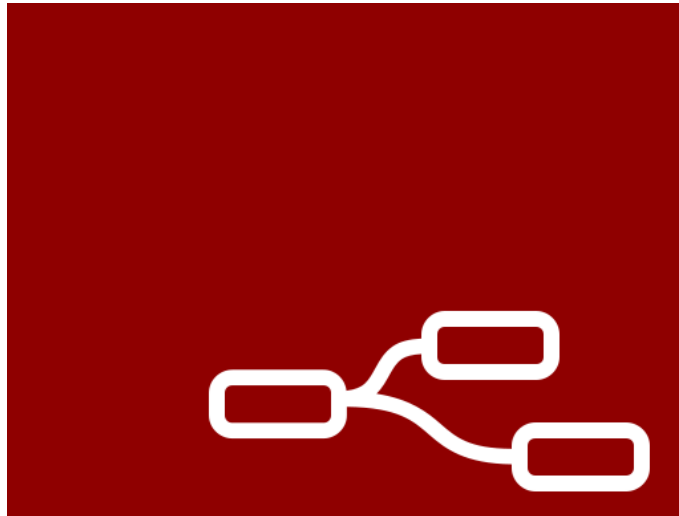


2.4 Uso de las herramientas HAL.



Caracteres ASCII de control			Caracteres ASCII imprimibles			
00	NULL	(carácter nulo)	32	espacio	64	@
01	SOH	(inicio encabezado)	33	!	65	A
02	STX	(inicio texto)	34	"	66	B
03	ETX	(fin de texto)	35	#	67	C
04	EOT	(fin transmisión)	36	\$	68	D
05	ENQ	(consulta)	37	%	69	E
06	ACK	(reconocimiento)	38	&	70	F
07	BEL	(timbre)	39	'	71	G
08	BS	(retroceso)	40	(	72	H
09	HT	(tab horizontal)	41	)	73	I
10	LF	(nueva línea)	42	*	74	J
11	VT	(tab vertical)	43	+	75	K
12	FF	(nueva página)	44	,	76	L
13	CR	(retorno de carro)	45	-	77	M
14	SO	(desplaza afuera)	46	.	78	N
15	SI	(desplaza adentro)	47	/	79	O
16	DLE	(esc.vínculo datos)	48	0	80	P
17	DC1	(control disp. 1)	49	1	81	Q
18	DC2	(control disp. 2)	50	2	82	R
19	DC3	(control disp. 3)	51	3	83	S
20	DC4	(control disp. 4)	52	4	84	T
21	NAK	(conf. negativa)	53	5	85	U
22	SYN	(inactividad sínc)	54	6	86	V
23	ETB	(fin bloque trans)	55	7	87	W
24	CAN	(cancelar)	56	8	88	X
					120	x

Node-RED



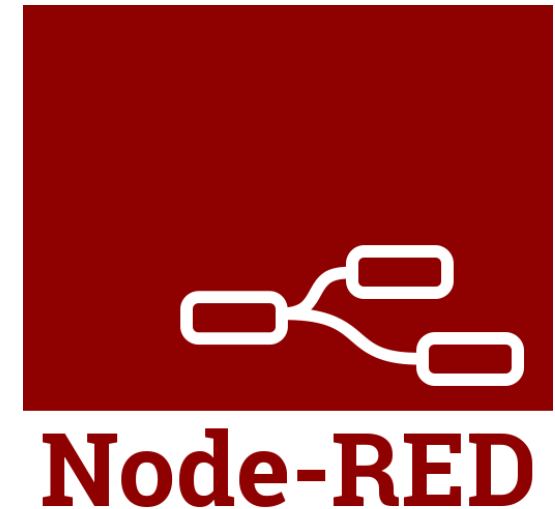
2.5 Node-RED.

4.1- Node-RED.

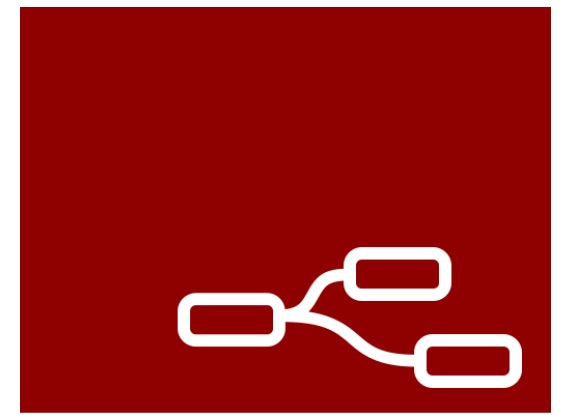
4.2- Instalación.

4.3- Práctica: Nodo Inject, Debug y Function.

4.4- Nodo Serie.



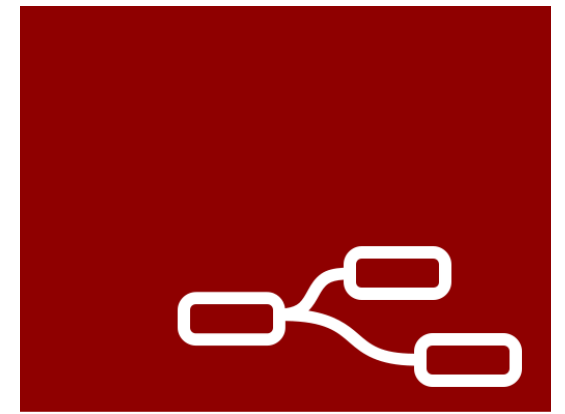
2.5 Node-RED.



Node-RED

- **Herramienta de programación visual** que permite conectar dispositivos, APIs y servicios online de forma sencilla y rápida.
- Si bien se puede escribir código, generalmente **se utilizan bloques funcionales llamados nodos, que se unen para crear flujos de trabajo.**
- Estos flujos definen la lógica de la aplicación, cómo los datos se mueven y qué acciones se disparan.
- Construido sobre **Node.js**, un entorno de ejecución de JavaScript ligero, eficiente y capaz de manejar una gran cantidad de conexiones.

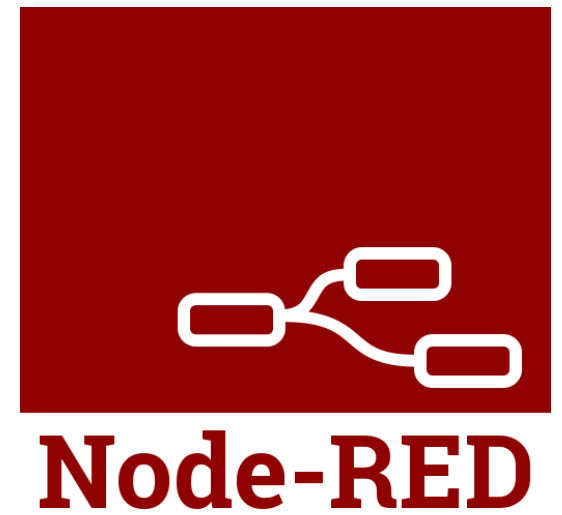
2.5 Node-RED.



Node-RED

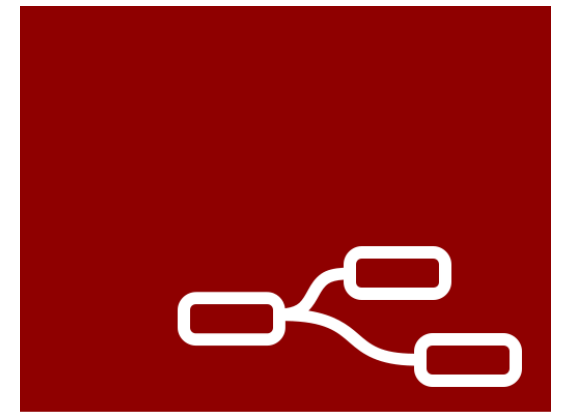
- Nació en 2012 en el Laboratorio Hursley de IBM (R.U.), **como una herramienta para que desarrolladores implementen rápidamente aplicaciones para IoT.**
- Objetivo: simplificar procesos, dada su utilidad y la creciente demanda de herramientas de integración en el ecosistema IoT.
- IBM liberó Node-RED como un proyecto **Open Source** en 2013, decisión clave para alcanzar su popularidad (hoy la mantiene la OpenJS Foundation).

2.5 Node-RED.



- Filosofía: reducir la cantidad de código para un desarrollador, la lógica se construye conectando nodos.
- No es un "no-code" puro, se puede usar el **nodo function** para escribir pequeñas porciones de código JavaScript, sin tener que construir toda la aplicación desde cero con código.
- Para ingenieros y técnicos, esto significa poder realizar integraciones avanzadas sin ser expertos en programación.

2.5 Node-RED.

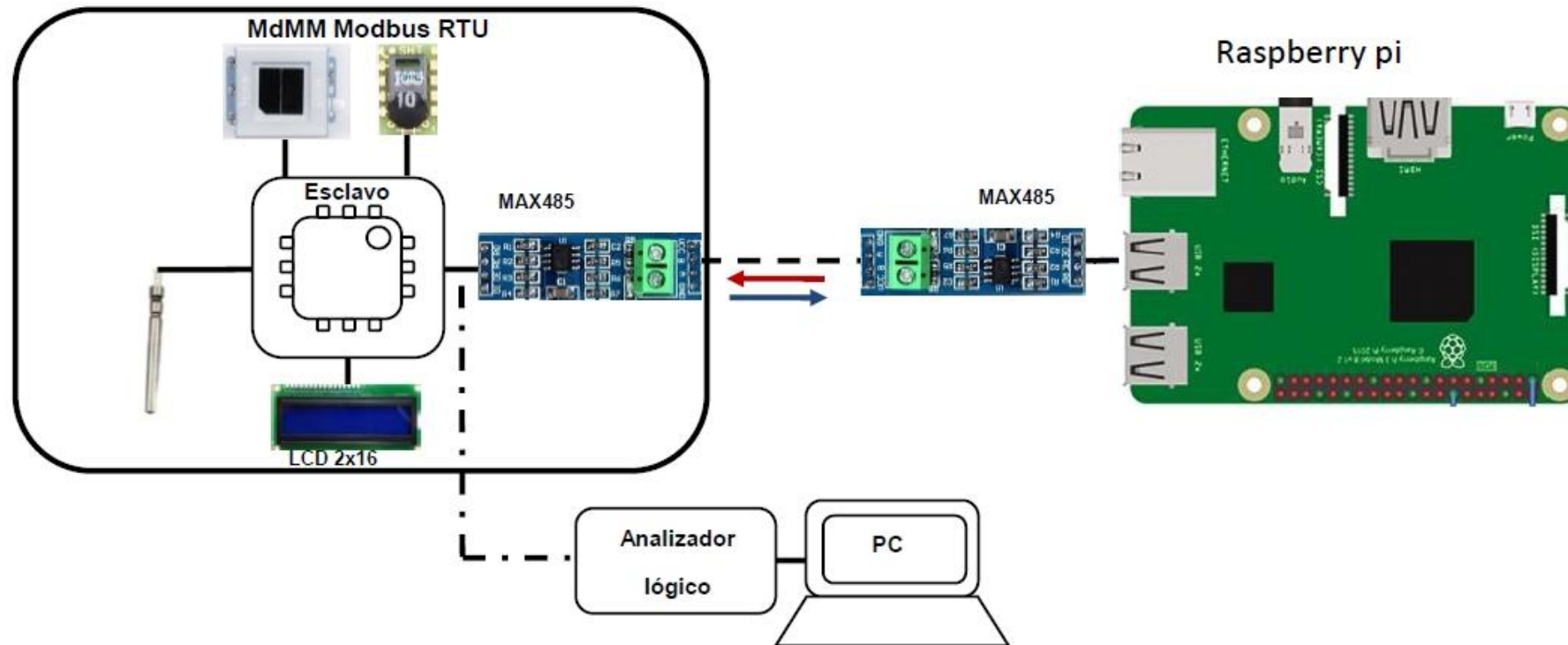
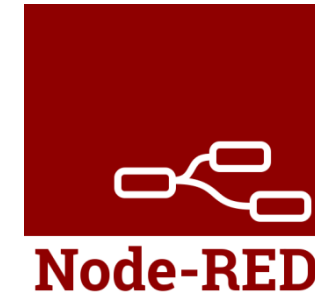


Node-RED

- **No-Code:** Algunas tareas se realizan sin escribir una sola línea de código, **ej: recibir datos de un sensor MQTT y mostrarlos en un debug.**
- Puedes construir y probar aplicaciones en minutos u horas.
- La interfaz visual abstrae gran parte de la complejidad de la programación, permitiéndote concentrarte en la lógica o el proceso.
- Los nodos facilitan la conexión con cualquier sistema, bases de datos, APIs REST, servicios cloud, hardware, redes Modbus, y por supuesto, sistemas IoT y redes LoRaWAN.

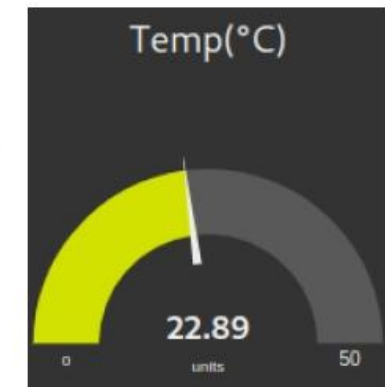
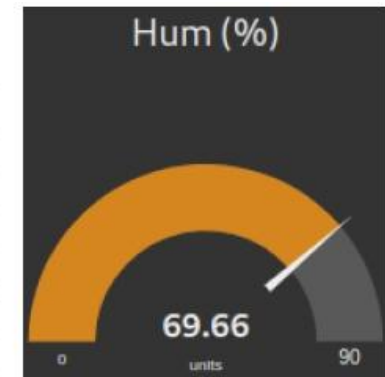
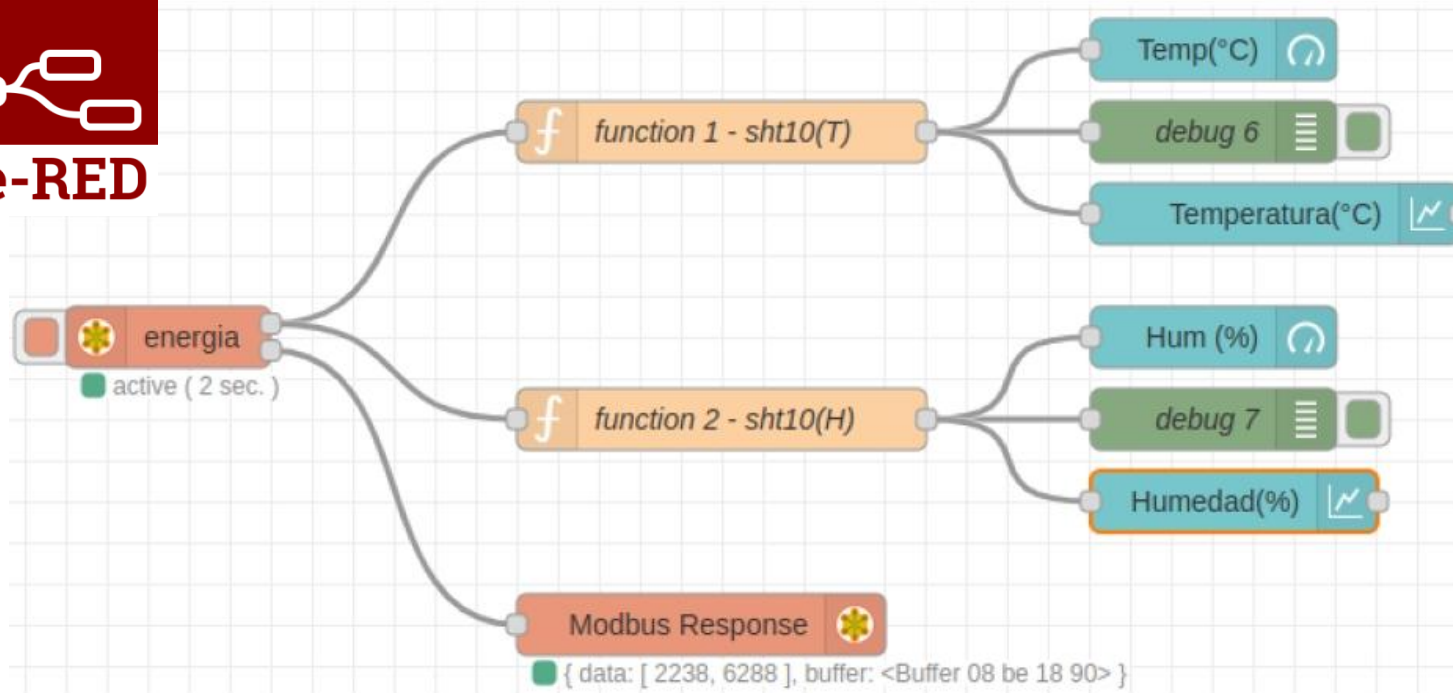
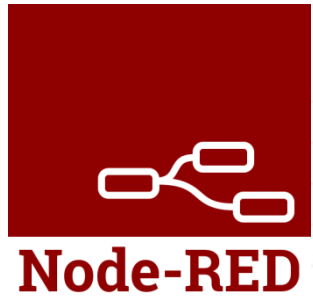
2.5 Node-RED.

- Ejemplos de aplicación: Modbus RTU.



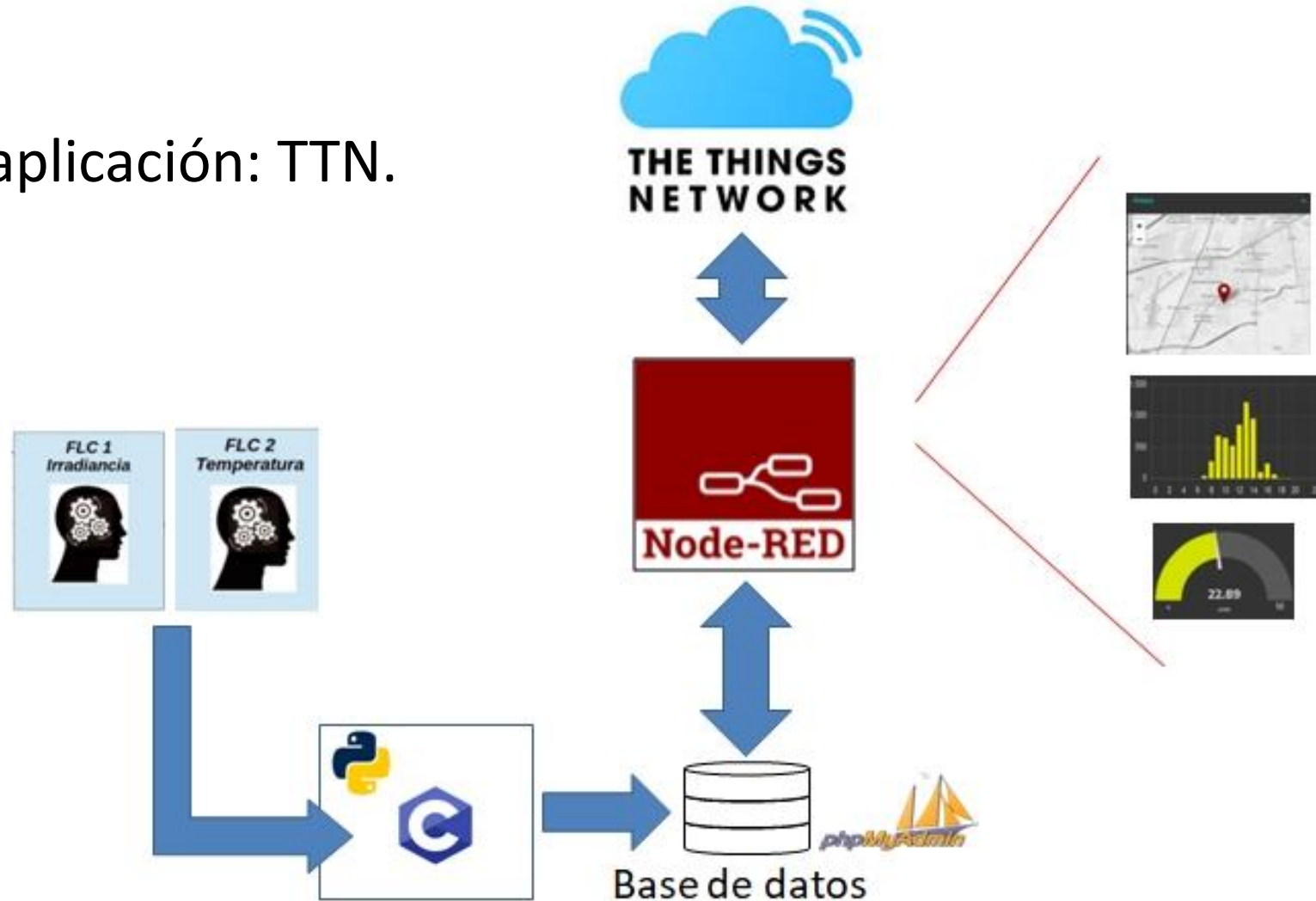
2.5 Node-RED.

- Ejemplos de aplicación: Modbus RTU.



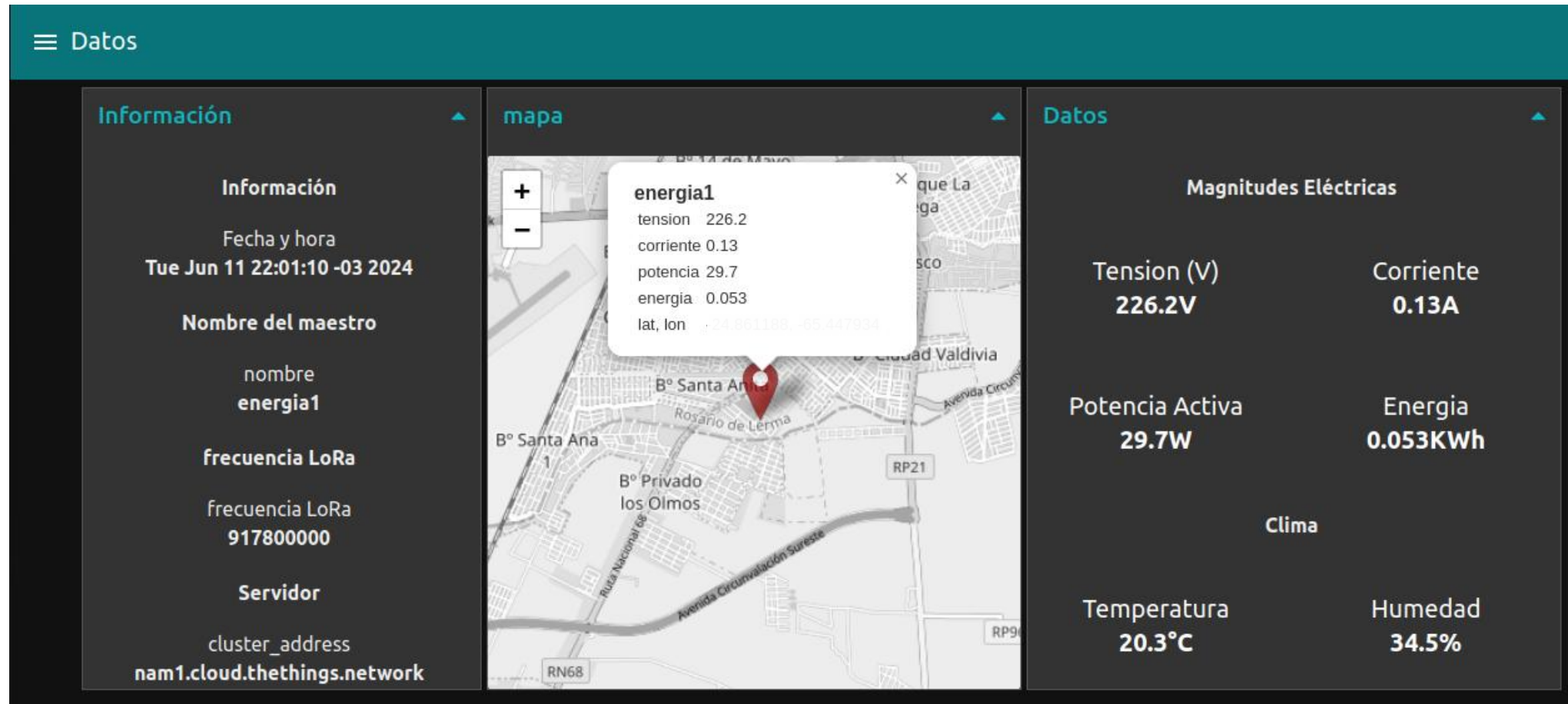
2.5 Node-RED.

- Ejemplos de aplicación: TTN.



2.5 Node-RED.

- Ejemplos de aplicación: TTN.

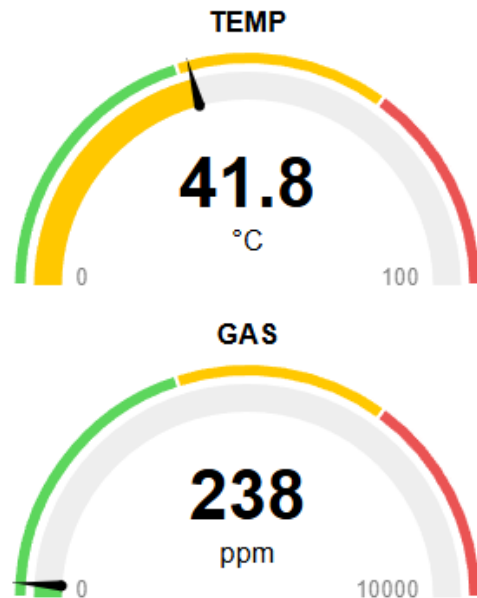


2.5 Node-RED.

- Dashboard para Example 1.1.

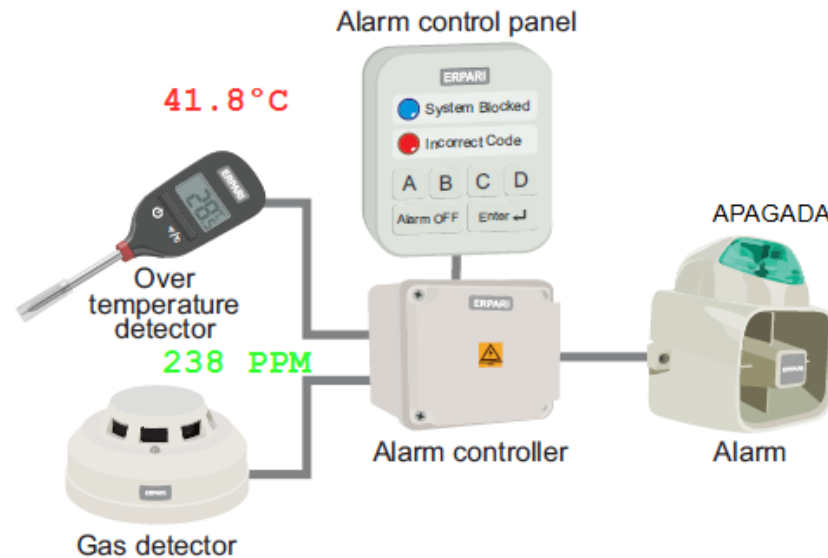
≡ Node-RED y NUCLEO-F446

PruebaADC



Ref: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers

Ejemplo1-1



4.5.1 Instalación de Node-RED

- Instalación local de Node.js (Windows/macOS/Linux).

<https://nodejs.org/es>

- Se instala aquí también el gestor de paquetes npm.

<https://www.npmjs.com/>

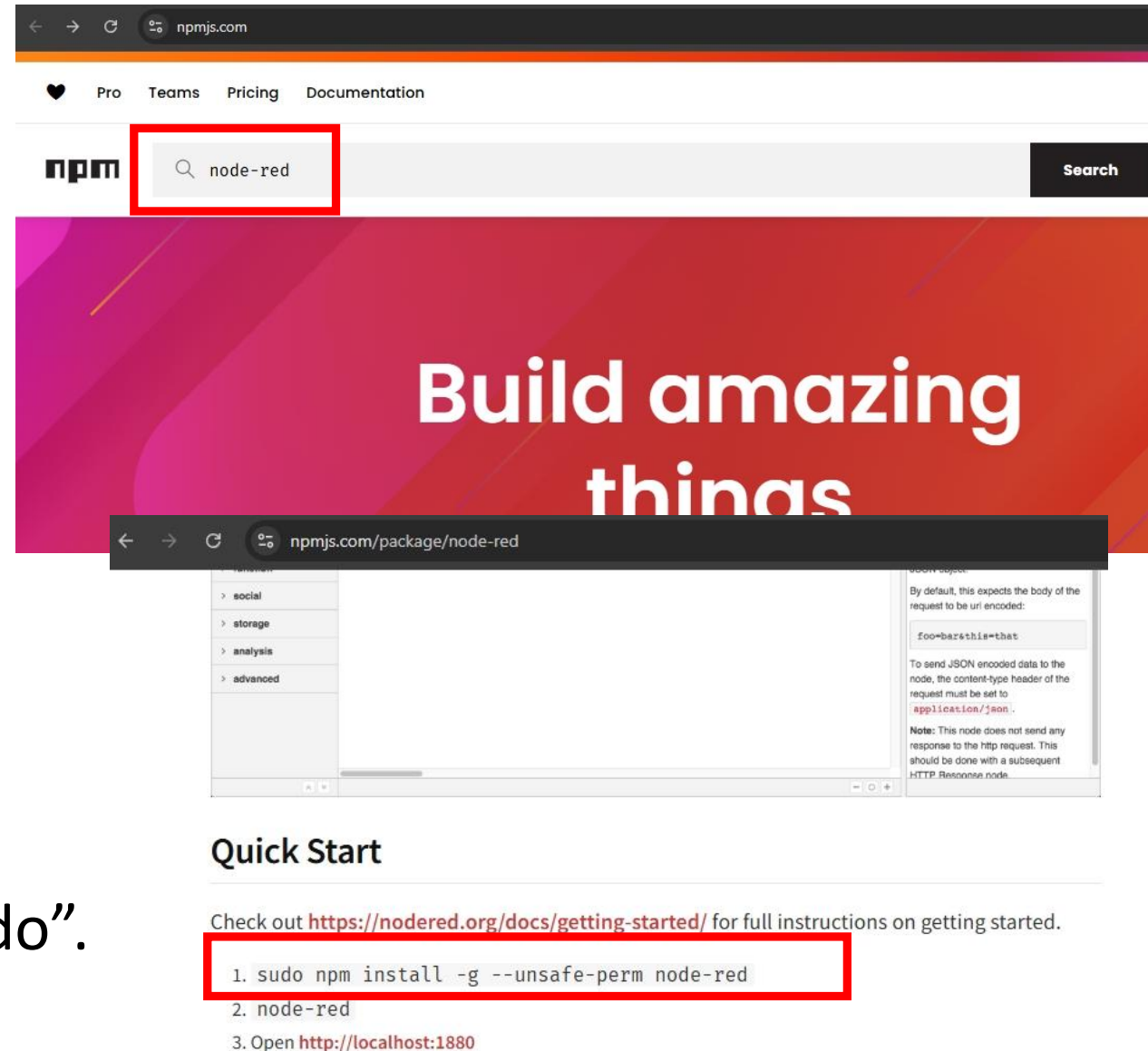


4.5.1 Instalación de Node-RED

- En la el enlace se puede buscar información acerca de la instalación de Node-RED.

<https://www.npmjs.com/>

- Indica el comando para instalar desde windows.
- Comando desde consola sin “sudo”.
- Ejecutar: node-red



4.5.1 Instalación de Node-RED

The screenshot displays the Node-RED web interface. The top bar includes the Node-RED logo and a red button labeled "Instanciar". Below the bar, a search bar "filtrar nodos" is present. The left sidebar shows a list of nodes under the "común" (common) category, including inject, debug, complete, catch, status, link in, link call, link out, and comment. The "función" (function) category is also visible. The main workspace, titled "Flow 1", contains a flow diagram with the following components: an "entrada1" (inject) node, a "debug 2" node, a "function 1" node, a "gauge" node, a "debug 1" node, a "debug 3" node, and a purple node labeled "# conectado". The flow starts with "entrada1" connected to "debug 2". "function 1" is connected to "gauge", "debug 1", and "debug 3". The "# conectado" node is connected to "function 1". The right sidebar shows the "depuración" (debug) console, which displays messages from the debug nodes. The messages include object data for "debug 1" and "debug 3", and a number "1.5" for "debug 1".

Node-RED

Instanciar

filtrar nodos

Flow 1

común

- inject
- debug
- complete
- catch
- status
- link in
- link call
- link out
- comment

función

depuración

todos los nodos

todo

```
{ end_device_ids: object, correlation_ids: array[1], received_at: "2025-06-05T21:45:19.747648852Z", uplink_message: object }
```

5/6/2025, 18:45:20 nodo: debug 1
v3/arduino-sx1276@ttn/devices/mi-arduino-sx1276/up :
msg.payload : number
1.5

5/6/2025, 18:45:30 nodo: debug 3
v3/arduino-sx1276@ttn/devices/mi-arduino-sx1276/up :
msg.payload : Object
{ end_device_ids: object, correlation_ids: array[1], received_at: "2025-06-05T21:45:29.759301139Z", uplink_message: object }

5/6/2025, 18:45:30 nodo: debug 1
v3/arduino-sx1276@ttn/devices/mi-arduino-sx1276/up :
msg.payload : number
1.5

2.5.2 Nodo Inject. Parámetros.



- Se utiliza para iniciar un flujo del mensaje ("botón" o "temporizador").

Payload: Contenido principal del mensaje (puedes configurarlo).

Timestamp: envía la marca de tiempo actual en ms, desde el 1 de enero de 1970. Es útil para saber cuándo se activó un flujo.

String: Una cadena de texto.

Number: Un valor numérico.

Boolean: true o false.

2.5.2 Nodo Inject. Parámetros.



Payload: Contenido principal del mensaje, puedes configurarlo.

Buffer: Datos binarios.

JSON: Un objeto JSON, útil para enviar datos estructurados.

JSONata: Permite construir payloads dinámicos utilizando JSONata.

Flow/Global context: Puedes inyectar el valor de una variable de contexto (ya sea del flujo actual o global).

Topic: Cadena de texto que se adjunta al mensaje como msg.topic.

2.5.2 Nodo Inject. Parámetros.



Repeat: Aquí es donde configuras cómo se dispara el nodo inject:

None (manual): El nodo se activa solo cuando haces clic en su botón.

Interval: El nodo se activa repetidamente en un intervalo de tiempo.

At a specific time: El nodo se puede activar en un momento del día.

At intervals between times: Se activa a intervalos de tiempo.

Name: Nombre opcional para el nodo.

2.5.2 Nodo Debug. Parámetros.



- Es el nodo para la depuración y el monitoreo de flujos de datos, permite ver el contenido de los mensajes.

Output: define qué parte del mensaje se va a mostrar;

msg.payload: Muestra solo el contenido del payload.

Complete msg object: Muestra todo el objeto msg, es muy útil para entender la estructura completa del mensaje.

JSONata expression: Permite usar expresiones JSONata.

2.5.2 Nodo Debug. Parámetros.



Debug window: La salida aparece en el panel de depuración.

System console: La salida se envía a la consola donde se ejecuta Node-RED.

Node status: Muestra un pequeño resumen de la salida debajo del nodo en el propio editor.

Name: Un nombre opcional para el nodo.

2.5.2 Nodo Debug. Parámetros.



Debug window: La salida aparece en el panel de depuración.

System console: La salida se envía a la consola donde se ejecuta Node-RED.

Node status: Muestra un pequeño resumen de la salida debajo del nodo en el propio editor.

Name: Un nombre opcional para el nodo.

2.5.3 Ejemplo



- Representar el flujo de un mensaje de un sensor de temperatura:

Payload, número igual a 37,5°C.
Topic: /sensor/temperatura.

Propiedades

Nombre: entrada1

msg. payload = 37.5

msg. topic = /sensor/temperatura

2.5.3 Ejemplo



- Representar el flujo de un mensaje de un sensor de temperatura:

Payload, número igual a 37,5°C.

Topic: /sensor/temperatura.

Salida: mensaje completo.



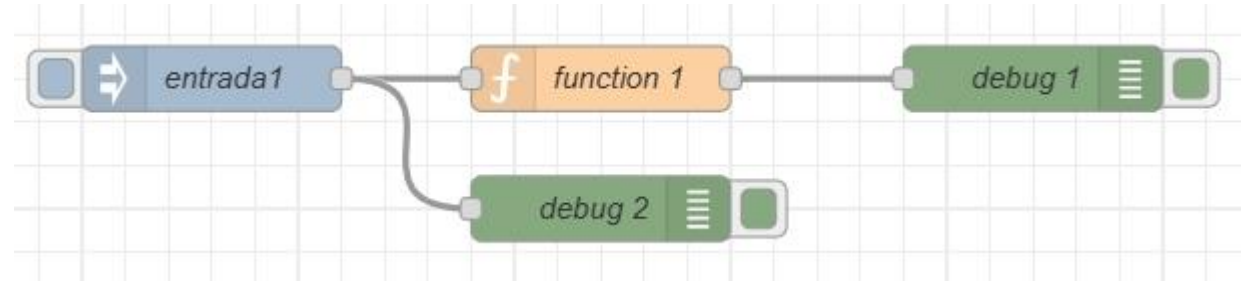
```
5/6/2025, 0:29:14  nodo: debug 1
/sensor/temperatura : msg : Object
▼object
  _msgid: "8a8573677640c95e"
  payload: 37.5
  topic: "/sensor/temperatura"
```

Salida: msg.payload



```
5/6/2025, 0:32:33  nodo: debug 1
/sensor/temperatura : msg.payload : number
37.5
```

2.5.3 Ejemplo



- Representar el flujo de un mensaje de un sensor de temperatura:

Payload, número igual a 37,5°C.

Topic: /sensor/temperatura.

⚙️ Propiedades

🔖 Nombre

function 1

⚙️ Configuración

Al inicio

En mensaje

```
1 var temp = msg.topic+" -> "+msg.payload+" °C";
2 msg.payload=temp;
3 return msg;
```

5/6/2025, 1:07:59 nodo: debug 2

/sensor/temperatura : msg : Object

▼ object

_msgid: "3cba07c3bce164af"

payload: 37.5

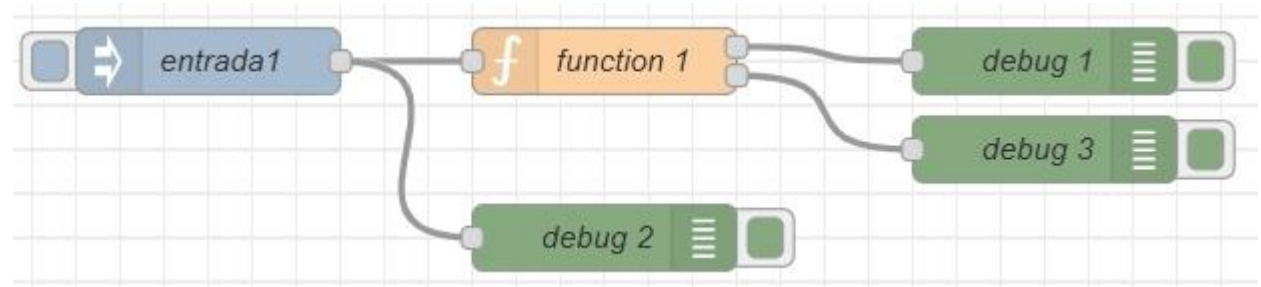
topic: "/sensor/temperatura"

5/6/2025, 1:07:59 nodo: debug 1

/sensor/temperatura : msg.payload : string[30]

"/sensor/temperatura -> 37.5 °C"

2.5.3 Ejemplo



- Representar el flujo de un mensaje de un sensor de temperatura:

Payload, número igual a 37,5°C.

Topic: /sensor/temperatura.

⚙️ Propiedades

📄 Nombre

function 1

⚙️ Configuración

Al inicio

En mensaje

Al final

```
1 var temp = {payload: msg.topic+" -> "+msg.payload+" °C"};
2 var temp2= {payload: msg.payload+" °C"};
3
4 return [temp,temp2];
```

5/6/2025, 1:23:01 nodo: debug 2

/sensor/temperatura : msg : Object

▼ object

_msgid: "89f26d1f8472ed1d"

payload: 37.5

topic: "/sensor/temperatura"

5/6/2025, 1:23:01 nodo: debug 1

msg.payload : string[30]

"/sensor/temperatura -> 37.5 °C"

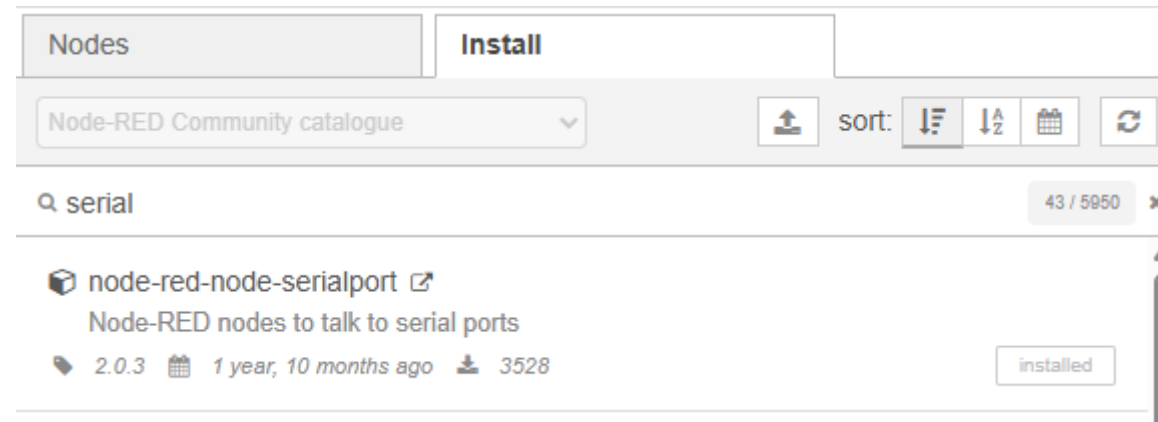
5/6/2025, 1:23:01 nodo: debug 3

msg.payload : string[7]

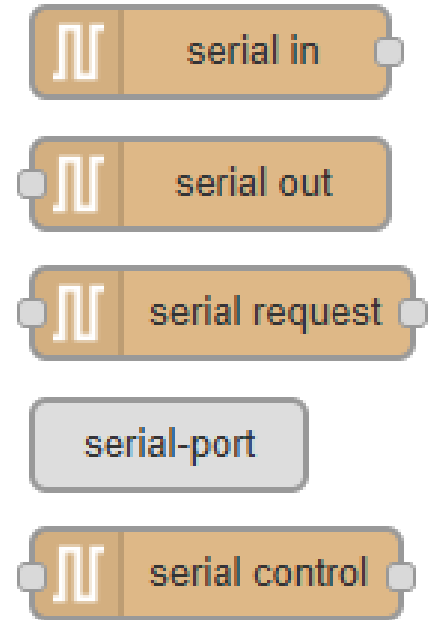
"37.5 °C"

2.5.4 Puerto Serie con Node-RED

- Instalar: `npm i node-red-node-serialport`

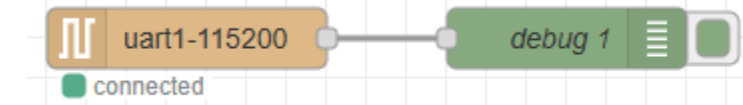


- Proporciona cuatro nodos: uno para recibir mensajes y otro para enviarlos; un nodo de solicitud que puede enviar mensajes y esperar una respuesta; y un nodo de control que permite el control dinámico de los puertos en uso.



2.5.4 Puerto Serie con Node-RED

- Serial in: Lee datos desde un puerto serie local.
- Espera un carácter de "separación" (por defecto \n).
- Control: Cuando se inicia Node-RED, el programa selecciona el puerto serie preprogramado, lo abre e inicia la comunicación. En algunos casos se debe cambiar el puerto.



Edit serial in node > Edit serial-port node

Delete Cancel Update

Properties

Name: uart1-115200

Serial Port: COM3

Settings

Baud Rate	Data Bits	Parity	Stop Bits
115200	8	None	1

DTR	RTS	CTS	DSR
auto	auto	auto	auto

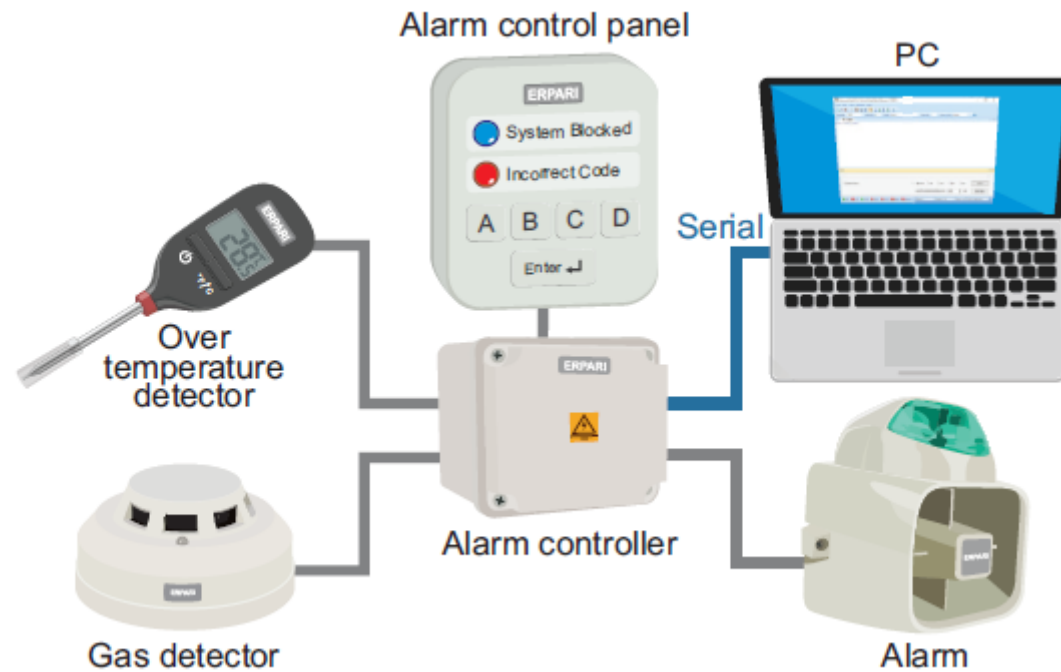
Input

Optionally wait for a start character of , then

Split input: on the character

and deliver: ASCII strings

Problemas



2.6 Retomamos el Example 1.3.

- En el problema se definen cuatro pines, tres como entrada y uno como salida. Las entradas representan la salida digital un sensor de gas y otro de temperatura.

La salida representa la activación de una alarma.

El botón reset apaga la alarma si se acciono.

- En la figura se explica la lógica del problema.

Extractos del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers (pag.19)
<https://www.arm.com/resources/education/books/designing-embedded-systems>

```
1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8     DigitalIn alarmOffButton(BUTTON1);
9
10    DigitalOut alarmLed(LED1);
11
12    gasDetector.mode(PullDown);
13    overTempDetector.mode(PullDown);
14
15    alarmLed = OFF;
16
17    bool alarmState = OFF;
18
19    while ( true ) {
20
21        if ( gasDetector || overTempDetector ) {
22            alarmState = ON;
23        }
24
25        alarmLed = alarmState;
26
27        if ( alarmOffButton ) {
28            alarmState = OFF;
29        }
30    }
31 }
```

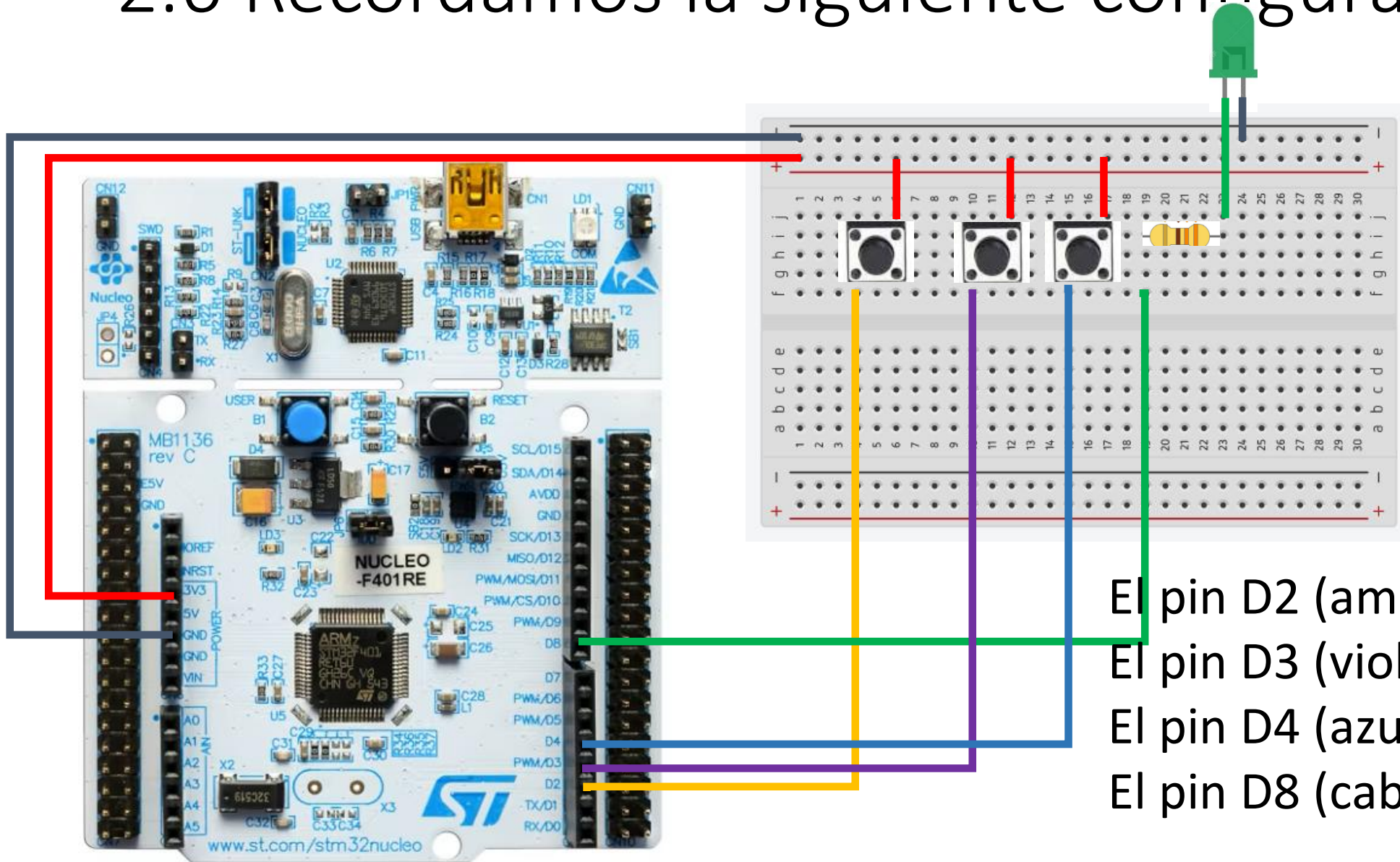
Include the libraries "mbed.h" and "arm_book_lib.h"

- Digital input objects named *gasDetector*, *overTempDetector* and *alarmOffButton*, are declared and assigned to D2, D3, and BUTTON1 (B1 USER), respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.
- *alarmLed* is assigned OFF.
- *alarmState* is declared and assigned OFF.

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, assign *alarmState* with ON.
- Assign *alarmLed* with *alarmState*.
- If *alarmOffButton*, assign *alarmState* with OFF.

2.6 Recordamos la siguiente configuración.



El pin D2 (amarillo) es PA10.

El pin D3 (violeta) es PB3.

El pin D4 (azul) es PB5.

El pin D8 (cable verde) es PA9.

2.6 Example 1.3 y agregamos la UART2

```
1 void uartTask()  
2 {  
3     char receivedChar = '\0';  
4     if( uartUsb.readable() ) {  
5         uartUsb.read( &receivedChar, 1 );  
6         if ( receivedChar == '1' ) {  
7             if ( alarmState ) {  
8                 uartUsb.write( "The alarm is activated\r\n", 24 );  
9             } else {  
10                uartUsb.write( "The alarm is not activated\r\n", 28 );  
11            }  
12        } else {  
13            availableCommands();  
14        }  
15    }  
16 }
```

Code 2.10 Details of the function `uartTask()`.

```
1 void availableCommands()  
2 {  
3     uartUsb.write( "Available command:\r\n", 20 );  
4     uartUsb.write( "Press '1' to get the alarm state\r\n\r\n", 36 );  
5 }
```

Code 2.11 Details of the function `availableCommands()`.

- Para el ejemplo agregamos las siguientes funciones.

Extractos del libro: A Beginner's Guide to Designing Embedded System Applications
on Arm Cortex-M Microcontrollers (pag.64)

<https://www.arm.com/resources/education/books/designing-embedded-systems>

2.6 Example 1.3 y agregamos la UART2

```
#include <string.h>
#include <stdbool.h>
#include "main.h"
#include "usart.h"
#include "gpio.h"
void SystemClock_Config(void);
void uartTask(void);
void availableCommands(void);
bool alarmState=0;
int main(void){
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();
    HAL_GPIO_WritePin(alarmLed_GPIO_Port, alarmLed_Pin, GPIO_PIN_RESET);
    HAL_Delay(100);
```

```
while (1){
    if(HAL_GPIO_ReadPin(gasDetector_GPIO_Port,
gasDetector_Pin)||HAL_GPIO_ReadPin(overTempDetector_GPIO_Port,overTempDetector_Pin)){

        alarmState=1;
    }

    HAL_GPIO_WritePin(alarmLed_GPIO_Port,
alarmLed_Pin, alarmState);

    if(HAL_GPIO_ReadPin(alarmoffButton_GPIO_Port,
alarmoffButton_Pin)){

        alarmState=0;
    }

    uartTask();
}}
```

2.6 Example 1.3 y agregamos la UART2

```
void availableCommands(void){
char msg1[] = "Available command: Press '1' \r\n";

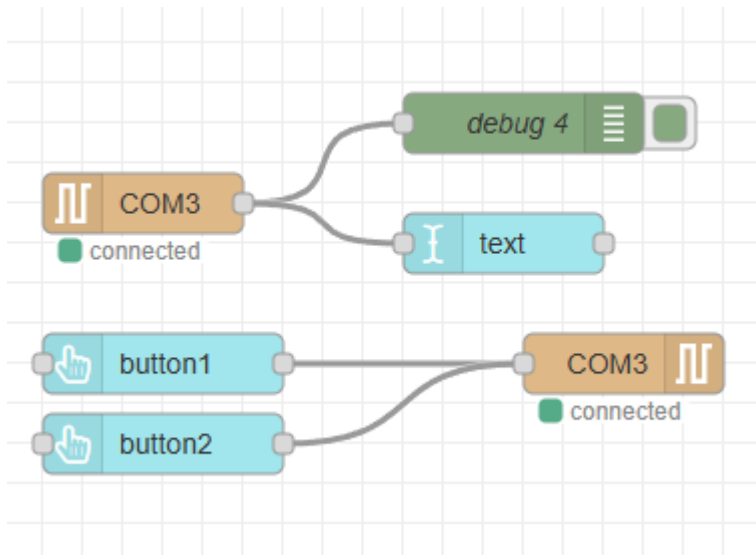
HAL_UART_Transmit(&huart2, (uint8_t*)msg1, strlen(msg1), 100);
}
```

```
void uartTask(void){
uint8_t receivedChar = '\0';

if (HAL_UART_Receive(&huart2, &receivedChar, 1, 10) == HAL_OK){
    if (receivedChar == '1'){
        if (alarmState){
            char msg[] = "The alarm is activated\r\n";
            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
        }else{
            char msg[] = "The alarm is not activated\r\n";
            HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
        } else {
            availableCommands();
        }
    }
}
```

2.6 Example 1.3 y agregamos la UART2

- En Node-RED realizaremos la siguiente aplicación.



≡ Page 2

Group 3

button2

text
The alarm is activated

button1

Bibliografia.

Lutenberg A., Gomez P. and Pernia E., 2022. A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Arm Education Media.

Ortiz J.P., Malagón P. J., Cárdenas R. and Gómez J.J., 2025. Fundamentos teóricos de sistemas basados en microcontrolador STM32, Sistemas Digitales II Sistemas Electrónicos, Universidad Politécnica de Madrid.

Cem Ünsalan C., Yücel M. E. and Gürhan H. D., 2018. Digital Signal Processing using Arm Cortex-M based Microcontrollers-Theory and Practice, Arm Education Media.

YIU J., 2019. System-on-Chip Design with Arm® Cortex® -M Processors, Arm Education Media